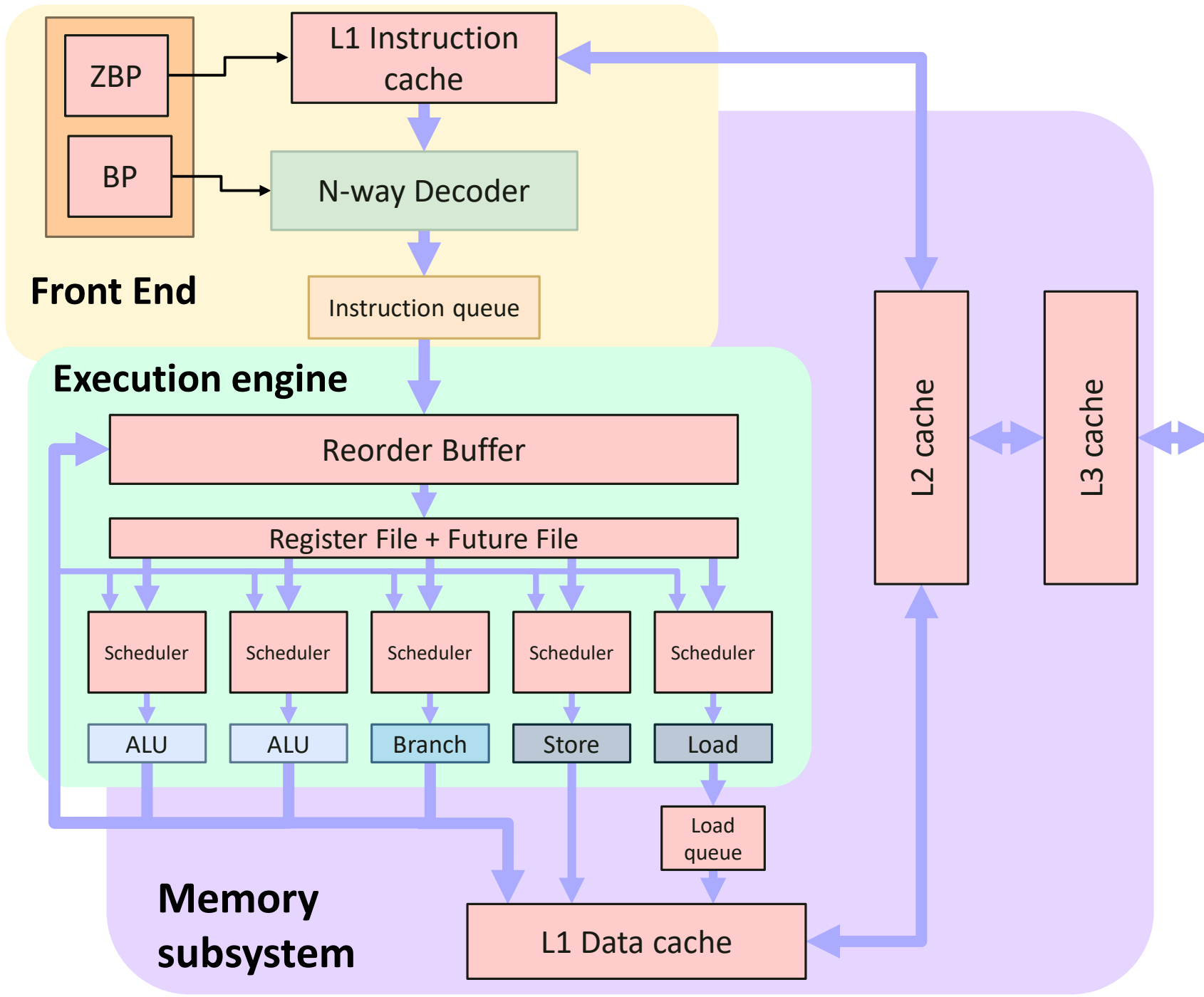
Superscalar processor simulator

BY MILOSZ WASACZ

Microarchitecture

A diagram of the simulated processor's microarchitecture

ZBP – Zero-bubble predictor
BP – Branch predictor



Features

- General
  - RISC-V unprivileged ISA
    - Support for all instructions in RV32I Base Integer ISA (except FENCE)
    - 32 GPRs, 32-bit
  - ELF file execution
- 8-stage pipeline
  - Zero-bubble prediction
  - Fetch
  - Branch prediction
  - Decode
  - Issue
  - Execute
  - Write result
  - Commit
- Pipeline features
  - Variable Fetch, Issue & Commit widths[1] (default: 4)
  - Variable size decode queue[1] (default: 16)
  - Speculative execution
  - Branch misprediction recovery
  - Precise exceptions
  - Variable ROB & Scheduler capacities (default: 96 & 16 per EU)[1]
- Execution Units
  - Four types: ALU, Branch, Load AGU, Store AGU
  - Multipurpose – a single EU can be of multiple types
- Branch prediction
  - Variable size zero-bubble predictor[1] (default: 1024)
  - Variable capacity 2-bit branch predictor[1] (default: 16K)
- Memory hierarchy[2]
  - Variable size, N-way associative L1I, L1D, L2 & L3 caches [1] (default: 4-way 32KiB, 4-way 32KiB, 8-way 256KiB & 16-way 2MiB)
  - Configurable write policies[1]
    - *Write through* & *Write back*
- Interactive environment
  - Full overview of the CPU state
  - Step, Run/Continue
  - Capturing *STDOUT* & *STDERR*

[1] Requires recompilation

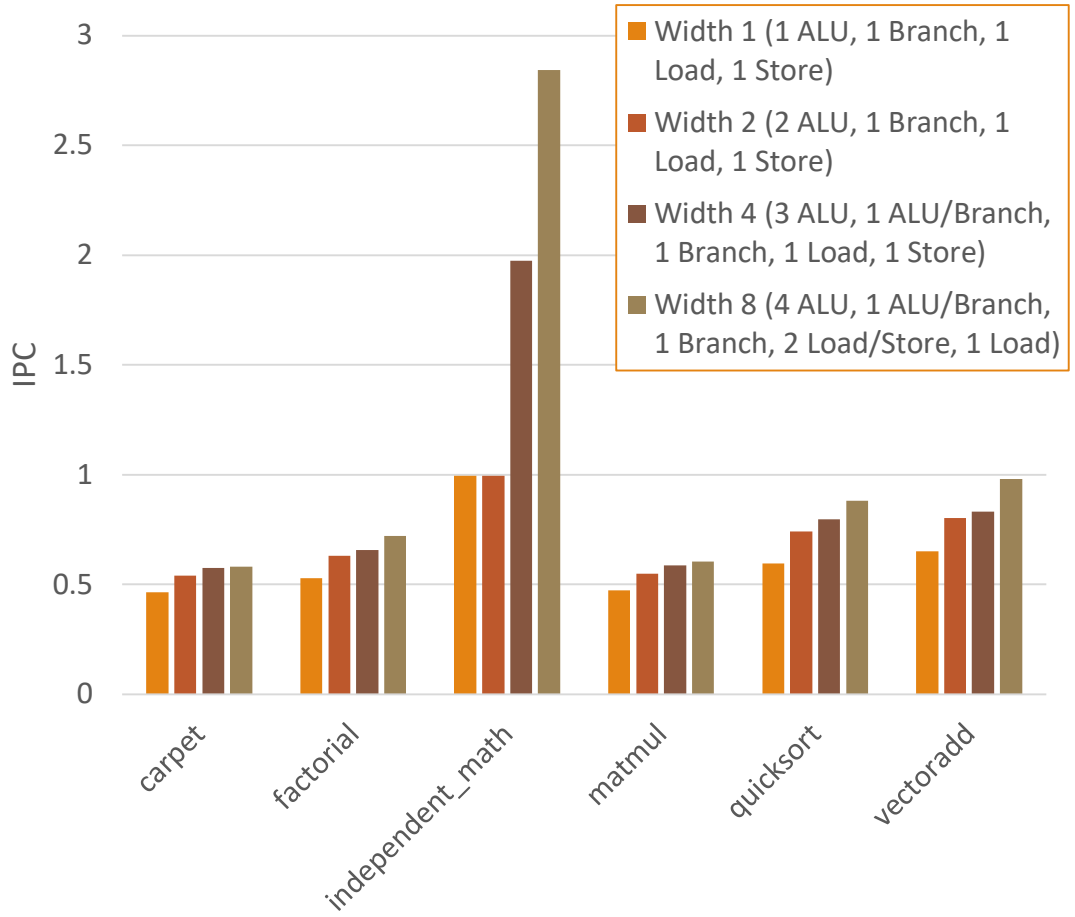
[2] Cache latencies not modelled

Benchmarks

carpet	Prints out 3 steps of Sierpiński carpet
factorial	Recursively calculates the factorial of 12
independent_math	Performs a 1000 iterations of a loop containing 4 completely independent additions
matmul	Multiplies two 5x5 matrices
quicksort	Sorts an array of 10 elements using the *quicksort* algorithm
vectoradd	Adds two 10-element vectors together

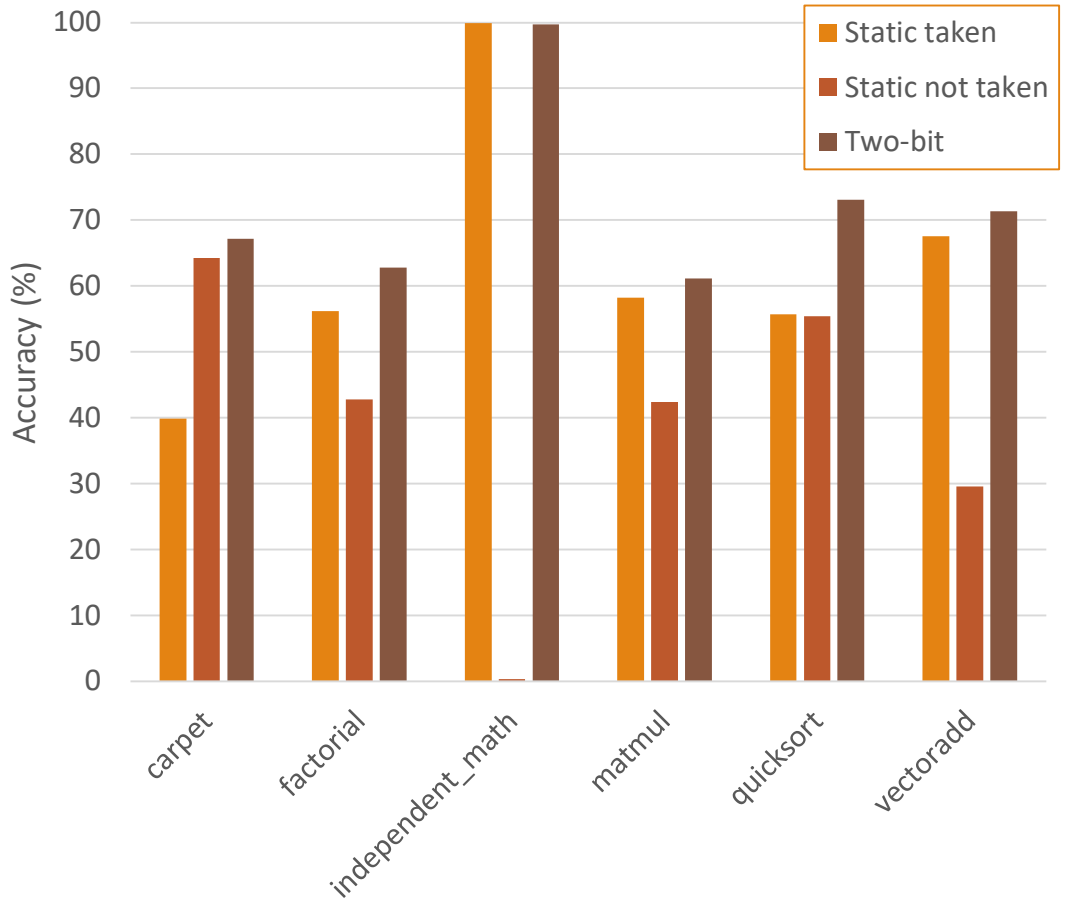
Experiment 1: Pipeline width

- **Hypothesis**
  - Increasing the pipeline width (Fetch, Issue, Commit & EU counts) will increase performance.
- **Experiment**
  - Processor performance was measured on several benchmark programs, comparing across 4 different pipeline configurations. Performance was measured by calculating the number of instructions executed per cycle.
- **Result**
  - It was found that increasing the pipeline width and the number of EUs available marginally increased the IPC, except for *independent_math*, for which the IPC increased almost 3 times between widths 1 and 8. This benchmark specifically has no memory operations, which introduce at least 1 extra cycle of latency than simple ALU operations.



Experiment 2: Branch prediction

- **Hypothesis**
  - Using a dynamic branch predictor yields better accuracy than using a static one.
- **Experiment**
  - Several benchmark programs were run on the processor with different branch prediction schemes enabled, comparing between two static ones (branch always taken & branch never taken) and a dynamic 2-bit predictor. Regardless of the employed scheme, unconditional jumps were always predicted to be taken. The accuracy was measured by calculating the ratio between correctly predicted branches and all branches executed.
- **Result**
  - It was found that a dynamic branch predictor was always better than a static not-taken one, and almost always better than a static taken one.
  - For the *independent_math* benchmark, a static taken branch predictor yielded a higher accuracy than the dynamic one – the former having a misprediction rate of 0.1%, whereas the latter having a misprediction rate of 0.3%. However, other benchmarks show that a dynamic branch predictor is far more versatile.



Experiment 3: Zero-bubble IPC

- **Hypothesis**
  - Using a zero-bubble predictor significantly increases performance.
- **Experiment**
  - Processor performance was measured on several benchmark programs, comparing across 6 different configurations, ranging in branch prediction scheme and whether the zero-bubble predictor was used. Performance was measured by calculating the number of instructions executed per cycle.
- **Result**
  - Apart from the carpet benchmark, all results show that employing a zero-bubble predictor increases the IPC; the increase, however, was marginal in most cases. The *independent_math* benchmark was again the only exception, achieving an IPC of almost 3 in a pipeline of width 4, more than double the IPC of the corresponding configurations without the ZBP.